

Programming Assignment 4

File Retrieval Engine, Client-Server, ZeroMQ Sockets

Instructions:

- Assigned date: Tuesday February 27th 2024
- Due date: 11:59pm on Tuesday March 12th 2024
- Maximum points: 100
- This programming assignment must be done individually.
- Please post your questions on the D2L Programming Assignments topic.
- Only the report must be uploaded to D2L. The code must be uploaded to the GitHub repository created for you for this assignment.
- Late submission will be penalized at 20% per late day.

This programming assignment aims to teach you about distributed systems architectures and processes, more precisely about the Client-Server Architecture and about inter-process communication through **ZeroMQ** sockets. As part of this homework assignment, you will need to implement a simple File Retrieval Engine that consists of two applications: a **client** and a **server**. The **server** application needs to support multiple **client** connections simultaneously and must provide indexing and search services for the all of the connected **clients**. The File Retrieval Engine must support indexing the files specified by the **clients** and must support executing queries over the indexed files, in order to return the most relevant files that match a client specified query. The query results must include the folders that the files reside in, as well as the clients that contain them. The programs that you need to implement as part of this assignment are limited to C++ and Java, the **ZeroMQ** socket library, and to the standard libraries. You can use any computer to develop the solution for this assignment, but you need to use a Linux machine, that runs Ubuntu 22.04, to evaluate the performance of your solution. The Ubuntu machine can be an instance from Chameleon Cloud or can be a Virtual Machine with at least 4 cores and 8GiB of RAM running on your personal computer.

1 File Retrieval Engine Architecture

The File Retrieval Engine must adhere to the architecture shown in Figure 1 and must consist of two applications: a **client** and a **server**. The overall File Retrieval Engine, must still be composed out of the three Application Layering components: **AppInterface**, **ProcessingEngine** and **IndexStore**; where the **ProcessingEngine** is distributed between the **client** and the **server** applications.

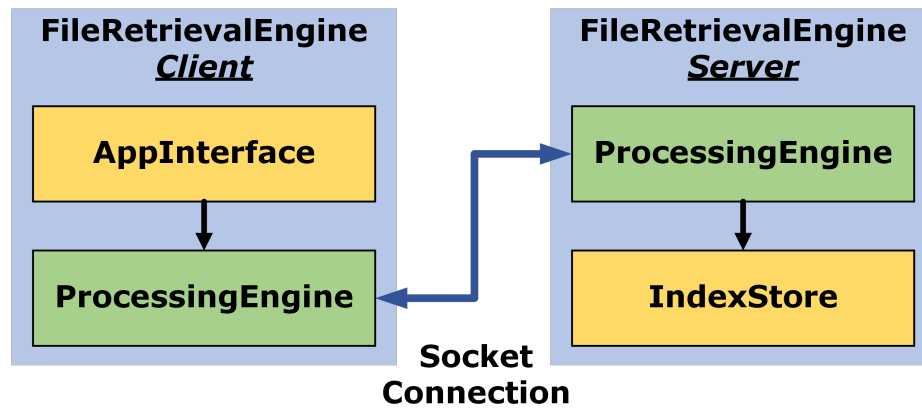


Figure 1: File Retrieval Engine Architecture.

The **AppInterface** component is responsible with implementing a command line interface that the user can use to interact with the File Retrieval Engine, with interpreting indexing and search commands submitted by the user, with forwarding the commands to the **ProcessingEngine** and with printing the results of the commands on screen. Both the **server** and the **client** have an **AppInterface** component.

On the **client** side, the **ProcessingEngine** is responsible with running indexing and search commands received from the **client AppInterface** component. In terms of indexing, the **ProcessingEngine** is responsible only with crawling the folder for files and, for each file, for extracting the words, for counting them and for sending them to the server to be inserted in the global index. In terms of search, the **ProcessingEngine** is responsible with extracting the words from a query and, for each word, for retrieving, from the server the documents and the frequencies that contain the word. After retrieving the lists of documents and frequencies for each word, the **client** side **ProcessingEngine** is responsible with combining the results using the intersection operations for documents and summation for frequencies, with sorting the combined results by frequency in descending order and with selecting the top 10 results.

On the **server** side, the **ProcessingEngine** is responsible with creating a dispatcher thread and with managing the worker threads created by the dispatcher thread. The dispatcher thread is responsible with creating a TCP/IP socket and for listening for new connections. When a new connection is detected, the dispatcher must call the **ProcessingEngine** to create a new worker thread to manage the new connection. The worker threads are responsible with receiving messages from the clients and with running either indexing or search operations. In terms of indexing, the a worker thread will receive an index message from a client containing the names of the folder and document and the list of words and their frequencies founds in the respective document, and must insert the words, document and frequency to the global index. In terms of search, the worker thread will receive a search message from a client with a single word and must return in another message the list of documents that contains the word as well as the number of occurrences.

The **IndexStore** component is responsible with storing the global index and providing the means to query the global index and resides only on the **server** side. More specifically, it should expose an interface to update the global index with a local index and an interface to

return the list of files and the number of occurrences in which a term can be found.

2 File Retrieval Engine Specification

The File Retrieval Engine Server must support the following commands:

- **quit:** this command closes the application by gracefully shutting down all of the active components (the dispatcher thread and all worker threads running as part of the application, while also closing all of the socket connections).
- **list:** this command lists all of the connected clients, including the IP address of the clients.

The File Retrieval Engine Client must support the following commands:

- **quit:** this command closes the application by gracefully disconnecting from the server.
- **connect <server IP and port >:** this command connects the client to the server.
- **index <dataset path>:** this command needs to tell the **client** side engine to crawl and find all of the files in the given folder path, and must extract and count the words of each file from the folder. Then it must construct an index message for each file containing the document path and the words and their occurrence and must send the message to the server in order to be inserted in the index. Sequences of alphanumeric characters (a-zA-Z0-9) that are separated by any other non-alphanumeric character(s) are considered as terms that need to be indexed.
- **search <AND query>:** this command needs to tell the **client** side engine to extract the terms from an AND query, and for each term construct a search message to be sent to the server. The server will construct a results message, containing the list of documents that contain a term and the frequency of the term in the documents. After the **client** side engine receives all of the results, it must combine the results using the intersection operations for documents and summation for frequencies, sort the combined results by frequency in descending order and it must selecting the top 10 results.

3 Datasets

You need to evaluate the performance of your solution over the [Gutenberg Project Datasets](#). There are 5 datasets with increasing size and number of files. Each dataset contains a number of 16 folder each containing a collection of ASCII TXT documents representing books collected from the Internet. You may test your program with other datasets, but your program must run without errors over this dataset to gain full points.

4 Indexing Performance Evaluation

You need to evaluate the performance of your solution with increasing number of clients (1, 2, 4, 8) for each dataset. For this you will need to submit in the report one plot and a discussion. The plot needs to capture the performance of your solution in terms of indexing only. For

indexing you need to measure the performance of each dataset in terms of number of files indexed per second (y-axis) with increasing number of clients (x-axis) for each dataset (5 lines/curves). The dataset needs to be distributed evenly between the clients. For example for a configuration of 4 clients, each client should index 4 different folder from the total number of 16 folders from a particular dataset.

Finally, you must include in your report a discussion about the interpretation of your evaluation, answering the following questions:

- What data structure did you use to implement the index(es) and why?
- How did you make sure that multiple clients were able to create a global index in a safe and correct way?
- How does the dataset size influence indexing performance?
- Why does your solution exhibit the performance that you measured?
- Is there anything that you could have done better?
- What was different when using ZeroMQ Sockets versus using POSIX Sockets?

5 Chameleon Cloud or Virtual Machine Testbed

You need to use a Virtual Machine running Ubuntu 22.04 with at least 4 cores and 8GiB of RAM, or the Chameleon Testbed to evaluate the performance of your application. If you are using Chameleon, when the code is ready and working correctly, spawn an instance on Chameleon Cloud and evaluate your code over the 5 datasets on that instance. The Chameleon instance needs to be of one of these types:

- `compute.icelake_r650` or `compute.icelake_r750`
- `compute.cascadelake` or `compute.cascadelake_r`
- `compute.skylake`
- `compute.haswell`

6 Where you will submit

You will have to submit your report to D2L and the code to a private git repository created for you for this programming assignment. The repository is created through GitHub Classroom and you will need to accept the assignment before you can clone it, by accessing this invitation link: <https://classroom.github.com/a/SoEIb0I2>. The first time you access the invitation link, the system will ask you to identify yourself. Please select the identifier that contains your school email address. After that you can clone the repository. Then you will have to add or update the source code and readme. Your submission on GitHub Classroom will not be graded unless you submit the report to D2L. The timestamp of your report on D2L will be used to determine if the submission is on time or late. If you cannot access your repository, please reach out at aorhean@depaul.edu.

7 What you will submit

As you work on the coding aspect of the assignment you should submit to the git repository each small functionality that you implement. When you have finished implementing the complete assignment you should submit your report to D2L. You need to hand in:

1. **Source code (50%):** All of the source code, including comments and a CMake build file for a C++ solution, or a Maven build file for a Java solution.
2. **README.md (10%):** A detailed manual describing the structure of your files and directory organization. The manual should contain a description of how to build the program and how to run the program.
3. **Report (40%):** A written PDF document describing the overall assignment completion, along with screenshots and text answers to the assignment questions. Make sure to label the answers with the appropriate part and question number.

Do not upload the datasets!

Submit step 1: code/build file/readme through git.

Submit step 2: report through D2L.

Grades for late submission will be lowered 20% per late day.